

CSE 232 : Data Structures

LAB 04 : LINKED LIST

Due Saturday at 5:00 p.m. MST

The fourth programming assignment of this semester is to write a class to implement linked lists using the pair programming technique.

Linked List

Please place your code in `node.h`. It will consist of a class with three methods.

- Default Constructor
- Copy Constructor
- Move Constructor

Additionally, there will be a collection of stand-alone functions.

- **Assign:** assign (copy the values), copy (duplicate an entire tree), and swap (swap root pointers).
- **Insert:** insert.
- **Remove:** remove and clear.
- **Status:** size.
- **Display:** The insertion operator.

Three Options

This week, there are three options to complete this lab.

85 Option : Integer Base Type

This will require you to implement all the above interfaces, but integer will be the base data-type. There are three member variables:

```
Node * pNext;           // pointer to next node
Node * pPrev;           // pointer to previous node
int data;
```

To complete the 85 option, please start with the following files: [232.04.Lab.85.zip](#).

100 Option : Template Base Type

This option will require you to implement all the same interfaces, but a template will be the base type.

```
template <class T>
```

There are three member variables:

```
Node * pNext;           // pointer to next node
Node * pPrev;          // pointer to previous node
T data;
```

To complete the 100 option, please start with the following files: [232.04.Lab.100.zip](#).

115 Option : Allocations

This option will require the class to everything from the 100 option. Additionally, it will be necessary to perform the specified number of allocations, deletions, etc. To complete the 115 option, please start with the following files: [232.04.Lab.115.zip](#).

As with last week, the 115 option requires a precise number of allocations and deletions. This will be measured with a `spy`. There is a video demonstration of the spy here: [Allocators and Spys](#). Note that the great advantage of the `std::allocator` is that we can independently allocate memory and call the constructor. If we use `new`, these happen together automatically. Since we only ever have one data element in each node, link lists (and binary trees in two weeks) do not need to perform these two actions independently. In other words, we can pass the 115 option without use of `std::allocator`.

Hint: The constructor of a class is guaranteed to call the constructor of all the member variables. If you do not provide an initialization section in your constructor, then C++ will call the member variable's default constructor. This may be what you want (and is most of the time). However, if our class's constructor is going to set the member variable to a specific value, it is more efficient to call the member variable's non-default constructor in the initialization section rather than allow the default constructor to be called followed by the assignment operator. There will be a unit test to exercise this exact case.

Assignment

Please do the following:

1. Select an option: 85, 100, or 115.
2. Download the code.
3. Implement the data structure with your partner(s).
4. Report how long it took for each individual, and what was the hardest part.
5. Upload your `node.h` code.

Assessment

The grade for your assignment will be based on the following criteria:

	Exceptional 100%	Good 90%	Acceptable 70%	Developing 50%	Missing 0%
Code Quality 35%	The most elegant solution was found for all the methods; there is no room for improvement,	There are no errors in efficiency, maintainability, cohesion, robustness, or redundancy.	There exists one error in efficiency, maintainability, cohesion, robustness, or redundancy.	The code quality is not what is expected for a CSE 232 student.	The code is sloppy or otherwise unprofessional.
Functionality 50%	Passes all the tests	Passes 90% of unit tests	Passes 70% of unit tests	Passes 50% of unit tests	Program fails to compile or no demonstration video was provided.

Template
Base Type
15%

The 100 or the
115 solution was
attempted

The 85 solution
was attempted

Allocations
15%

115 solution was
attempted

The 85 or the
100 solution was
attempted